StreamNet

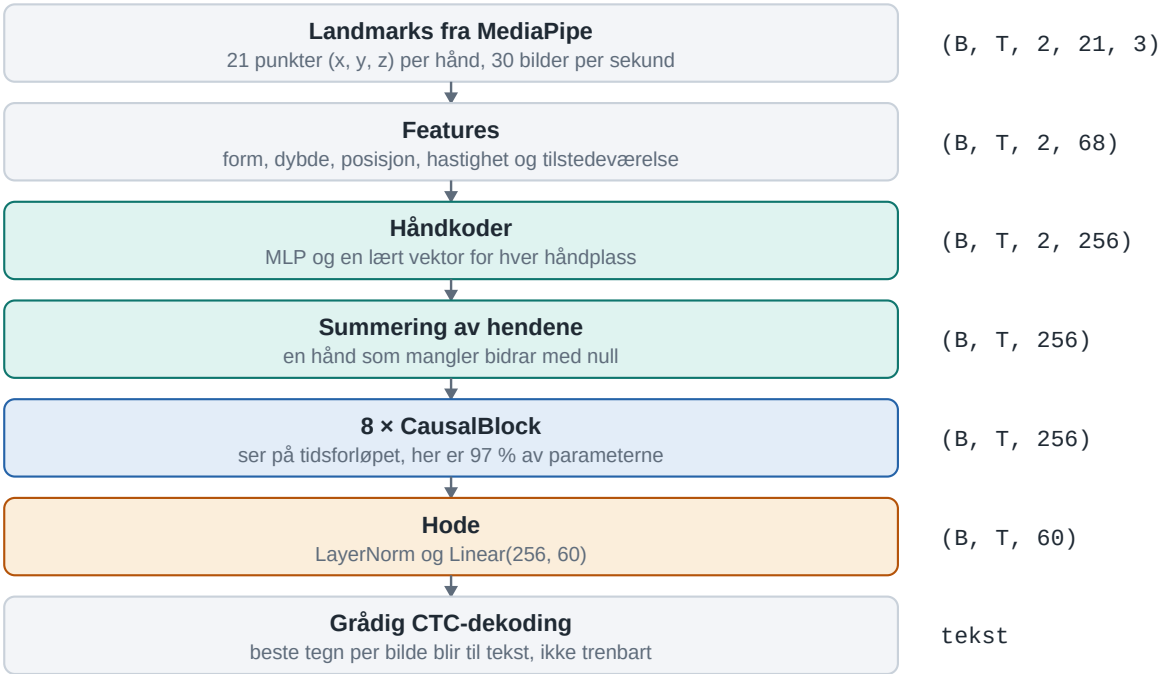
Nettverket som gjør håndbevegelser om til bokstaver i asl-shop · modell `bs40k` · 2. november 2026

StreamNet leser posisjonen til 21 punkter på hver hånd, 30 ganger i sekundet, og gir for hvert bilde et svar på hvilket tegn som vises akkurat nå. Det ser bare noen sekunder bakover og 0,3 sekunder fremover, og kan derfor kjøre direkte på et kamera.

Type	Kausalt, dilatert tidskonvolusjonsnett (TCN), trent med CTC
Inndata	To håndplasser med 21 punkter (x, y, z), gjort om til 68 features per hånd
Utdata	60 klasser per bilde: 59 tegn og en blank
Størrelse	3 276 860 parametere, 6,6 MB som float16
Kontekst	112 bilder bakover (3,7 s) og 9 fremover (0,3 s) ved 30 Hz
Fart	Omtrent 5,5 ms per bilde på CPU med 4 tråder
Treffsikkerhet	5,95 % tegnfeil (CER) på validering, med andre tegnere enn i treningen
Opphav	Trent fra bunnen av på FSboard, ikke pretrained

1. Oversikt

Figur 1 viser hele veien gjennom nettverket. **B** er antall klipp i en batch og **T** er antall bilder. Tallet 2 er de to håndplassene, høyre og venstre. T beholdes helt til slutten, slik at hvert inngangsbilde får sitt eget svar.



Figur 1. Nettverket fra landmarks til tekst, med formen på dataene etter hvert steg.

2. Inndata: fra punkter til features

Nettverket får aldri se selve kamerabildet, bare landmarks fra MediaPipe. Disse regnes om til 68 tall per hånd, slik at samme håndform gir omtrent samme tall uansett hvor stor hånden er i bildet. Håndstørrelsen er avstanden fra håndleddet til langfingerens grunnledd.

Del	Antall	Hvordan	Hvorfor
Form	42	xy relativt til håndleddet, delt på håndstørrelsen. x speiles for venstre hånd.	Beskriver håndformen. Speilingen lar begge hender dele mønstre.
Dybde	21	z relativt til håndleddet, delt på håndstørrelsen.	Skiller former som ser like ut rett forfra.
Posisjon	2	Håndleddets xy delt på håndstørrelsen, ganget med 0,1.	Hvor hånden er i bildet.
Hastighet	2	Posisjon nå minus posisjon i forrige bilde.	Gjør J og Z mulig å lære, siden de tegnes i lufta.
Til stede	1	Alltid 1 når hånden finnes.	Skiller en ekte hånd fra en nullvektor.

Håndkoderen

Hver hånd sendes gjennom samme lille MLP: `Linear(68, 256) → LayerNorm → GELU → Linear(256, 256)`. Deretter legges en lært vektor for håndplassen til, slik at nettverket vet om det er høyre eller venstre. Til slutt summeres de to hendene til én vektor på 256 tall per bilde. En hånd som mangler ganges med null, så nettverket håndterer null, én eller to hender uten å endre form.

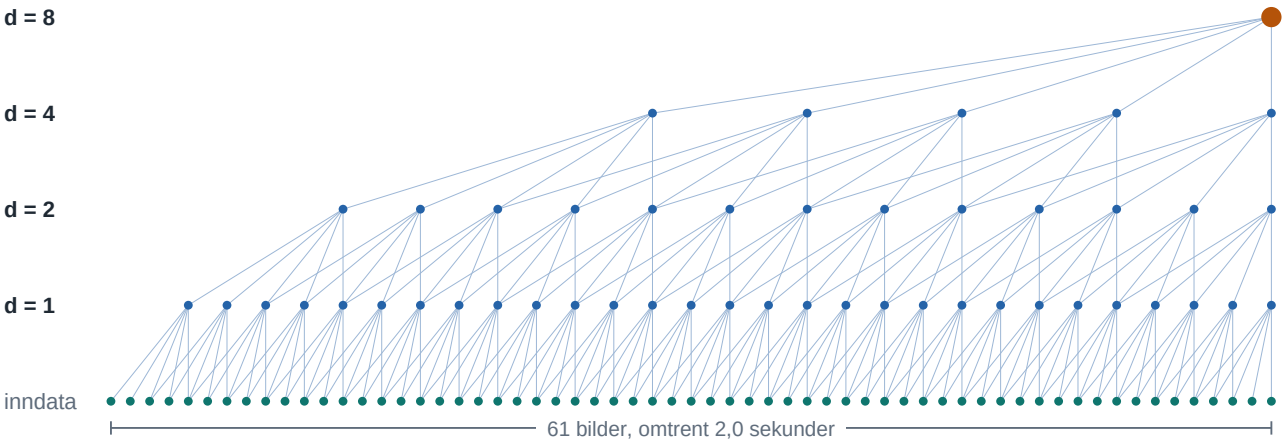
3. Tidsdelen: kausal og dilatert konvolusjon

En enkelt håndform er sjelden nok. Overganger og bevegelse er det som skiller tegnene, så nettverket må se flere bilder samtidig. Det bruker tre idéer:

Konvolusjon: et lite filter med 5 vekter glir langs tiden og kombinerer 5 bilder om gangen.

Kausal: filteret ser bare bakover. Det får nettverket til å passe til en strøm, der fremtiden ikke finnes ennå.

Dilatert: filteret hopper over bilder. Med dilasjon 8 leser det hvert åttende bilde. Hver blokk kan derfor dekke mye lengre tid uten at filteret får flere vekter.

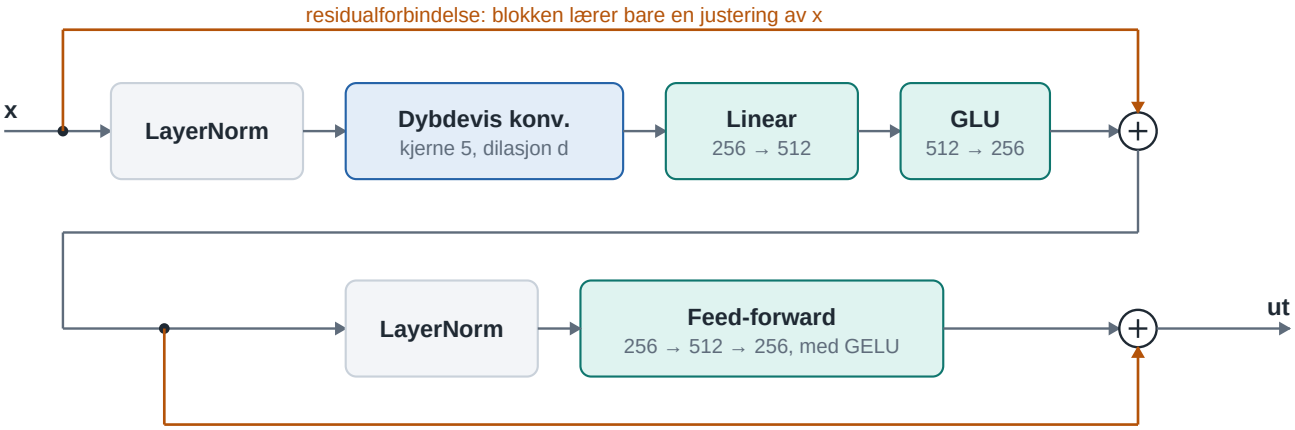


Figur 2. De fire første blokkene, med faktisk kjerne 5. Ett resultat (oransje) samler informasjon fra 61 inngangsbilder. Fremtidsforskyvningen fra kapittel 5 er utelatt.

Dilasjonene brukes i to runder: 1, 2, 4, 8, 1, 2, 4, 8. En blokk utvider synsfeltet med `(kjerne − 1) · dilasjon` bilder. Summert gir det `1 + 4 · 30 = 121` bilder.

Blokk	1	2	3	4	5	6	7	8
Dilasjon	1	2	4	8	1	2	4	8
Synsfelt (bilder)	5	13	29	61	65	73	89	121

4. Inne i en CausalBlock



Figur 3. Én blokk. Tidsdelen er den blå boksen, mens de grønne blander informasjon mellom kanalene.

Dybdevis konvolusjon gir hver av de 256 kanalene sitt eget tidsfilter. Det er svært billig: bare 1 536 av blokkens 397 056 parametere, altså 0,4 %. Kanalene møtes først i lagene etter.

GLU (gated linear unit) er en port. Det lineære laget lager 512 verdier som deles i to, `a` og `b`, og resultatet er `a · sigmoid(b)`. Nettverket lærer dermed hvor mye av hvert signal som skal slippe gjennom.

Residualforbindelsene legger inndata til resultatet. Da kan signalet og gradientene passere gjennom alle de åtte blokkene, og hver blokk lærer bare en justering.

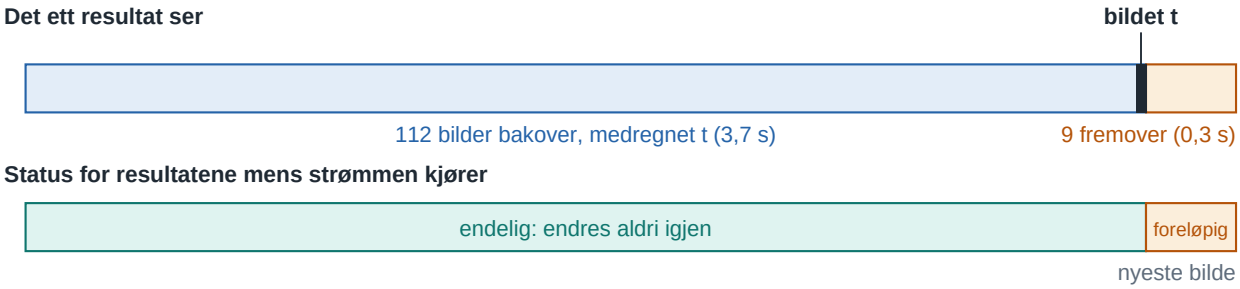
LayerNorm normaliserer hvert bilde for seg og bruker ingen statistikk over hele klippet. Det er viktig for strømming, siden hele klippet aldri er kjent. Under trening brukes også dropout på 0,1, som er slått av i appen.

Hvor parameterne er

Del	Parametere	Andel
Håndkoder	83 968	2,6 %
8 CausalBlock (397 056 hver)	3 176 448	96,9 %
Vektorer for håndplass og avsluttende LayerNorm	1 024	under 0,1 %
Hode, Linear(256, 60)	15 420	0,5 %
Totalt	**3 276 860**	**100 %**

5. Lookahead: hvordan den ser fremover

Med ren kausal konvolusjon ville nettverket måtte svare før det hadde sett bevegelsen ferdig. Derfor forskyves resultatene: nettverket får 9 tomme bilder på slutten av inndata, og de 9 første resultatene kastes. Resultatet for bilde `t` regnes altså ut når bilde `t + 9` er kommet. Nettverket er fortsatt kausalt, men svaret kommer 0,3 sekunder senere.



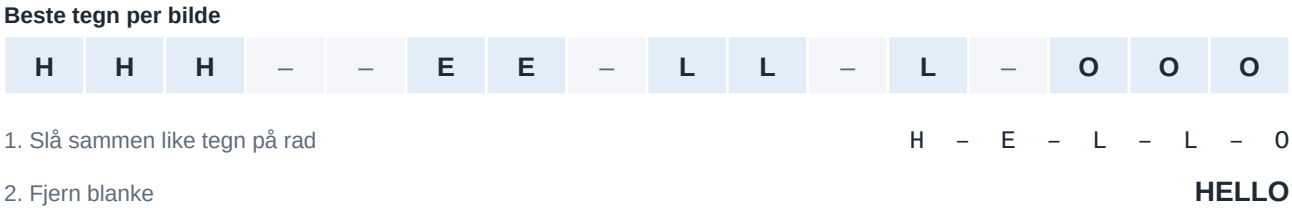
Figur 4. Øverst: de 121 bildene ett resultat bygger på. Nederst: de 9 nyeste resultatene regnes med fremtiden satt til «ingen hender», og kan derfor endre seg.

Siden resultatet avhenger av et fast vindu, regner appen bare om det nyeste vinduet for hvert nye bilde. Kostnaden vokser dermed ikke med øktens lengde.

Målt på den ferdige modellen: da ett enkelt bilde `f` ble endret, endret resultatene `f − 9` til `f + 112` seg, og ingen andre. Det er ett bilde mer enn de 121 i synsfeltet, fordi hastigheten også leser forrige bilde. Appens vindu gir eldste bilde hastighet null, så der blir det et lite avvik fra en kjøring over hele klippet.

6. Utdata: fra sannsynligheter til tekst med CTC

Hodet gir 60 tall per bilde. Ett av dem er **blank**, som betyr «ingen ny bokstav her». Treningsdataene sier bare hvilken tekst et klipp inneholder, ikke når hver bokstav skjer. CTC (connectionist temporal classification) løser dette ved å summere sannsynligheten for alle plasseringer av bokstavene som gir riktig tekst. I appen brukes grådig dekoding: beste tegn per bilde, som i figur 5.



Figur 5. Illustrasjon av grådig CTC-dekoding. Blank-tegnet mellom de to L-ene er det som gjør at de blir to bokstaver.

7. Trening og resultat

Data	FSboard-landmarks og menneskeskrevne transkripsjoner. Ingen pretrained vekter eller syntetiske klipp.
Tap og optimering	CTC-tap, AdamW med LR 2e-3 og weight decay 0,01, gradientklipping på 1,0.
Læringsrate	3 % lineær oppvarming, deretter cosinus over 40 000 steg med 40 000 bilder per steg.
Augmentering	Tidsstrekking 0,8–1,25x, speiling, rotasjon, skala, skjæring, pikselstøy og tilfeldig tapte bilder (opptil 20 %).
Krav til modellen	Høyst 300 bilder bakover, høyst 9 fremover og høyst 15 ms på CPU med 4 tråder.
Validering	12 015 klipp: 5,95 % tegnfeil totalt, 5,83 % i klipp med J eller Z, og 47,8 % av klippene helt riktige.

Dette er ikke målt ennå

Denne modellen er ikke kjørt på test-splitten; forrige kjøring (`bs80k`) fikk 6,12 %. Den er heller ikke verifisert med ekte webkamera. Tallene over gjelder FSboard-klipp, og et kamera kan gi dårligere resultat, blant annet på grunn av håndethet og speiling.

Kode: nettverket i `model/stream_net.py`, treningen i `autoresearch/train.py`.